Problem 2

This notebook solves the problem using:

- **Linear Regression**
- **Lasso Regression**
- **Ridge Regression**

1. Import Libraries

- numpy: for numerical operations
- pandas:to load and handle the dataset
- train_test_split: is used to divide the data into training, validation, and testing sets
- LinearRegression, Lasso, and Ridge: are the regression models used
- mean_squared_error and mean_absolute_error: are used for evaluation
- StandardScaler: is used to scale the inputt features before training

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Lasso, Ridge
from sklearn.metrics import mean_squared_error, mean_absolute_error
from sklearn.preprocessing import StandardScaler
```

2. Read the Dataset

This function reads the dataset provided in the lab assignment and stores it in a Pandas DataFrame. DataFrame: is a table-like structure with rows and columns, which makes it easy to inspect and manipulate the data.

```
def read_California_dataset() -> pd.DataFrame:
    df = pd.read_csv("California_Houses.csv")
    return df
df=read_California_dataset()
df.head()
```

	Median_House_Value	Median_Income	Median_Age	Tot_Rooms	Tot_Be
0	452600.0	8.3252	41	880	129
1	358500.0	8.3014	21	7099	1106
2	352100.0	7.2574	52	1467	190
3	341300.0	5.6431	52	1274	235

	Median_House_Value	Median_Income	Median_Age	Tot_Rooms	Tot_Be
4	342200.0	3.8462	52	1627	280

3. Split the Dataset into Training, Validation, and Testing Sets

This function divides the dataset into:

- 70% training
- 15% validation
- 15% testing
- `X` contains the input features
- `y` contains the target variable, which is : Median_House_Value
- The training set is used to train the models
- The validation set is used to choose the best hyperparameters
- The testing set is used for final evaluation

```
def split_data(df: pd.DataFrame, training_fractor: float = 0.7,
         validation_fractor: float = 0.15, testing_fractor: float = 0.15,):
    assert abs(training_fractor + validation_fractor + testing_fractor - 1.0)
         < 1e-6

    X = df.drop(columns=["Median_House_Value"])
    y = df["Median_House_Value"]

    np.random.seed(42)

    X_training, X_temp, y_training, y_temp = train_test_split(X, y,
         train_size=training_fractor)

    X_validation, X_testing, y_validation, y_testing = train_test_split(
        X_temp, y_temp, train_size=validation_fractor / (validation_fractor +
         testing_fractor)
    )

    return X_training, X_validation, X_testing, y_training, y_validation,
         y_testing
```

4. Train and Evaluate the Models

in this section:

1. We Scale the features using StandardScaler
2. Trains a Linear Regression model

3. Tries different `alpha` values for Lasso and Ridge.
4. Uses the validation set to choose the best `alpha`.
5. Evaluates all final models on the "test set" using:
    - MSE (Mean Squared Error)
    - MAE(Mean Absolute Error)

Why scaling is important

Scaling makes all features have comparable ranges. This is especially important for "Lasso" and "Ridge", because both use regularization penalties that depend on the size of the coefficients.

```
def train_and_evaluate_models(X_training, y_training, X_validation,
         y_validation, X_testing, y_testing):
    scaler = StandardScaler()

    X_training_scaled = scaler.fit_transform(X_training)
    X_validation_scaled = scaler.transform(X_validation)
    X_testing_scaled = scaler.transform(X_testing)

    print("--- Tuning Models on Validation Set ---")

    lr = LinearRegression()
    lr.fit(X_training_scaled, y_training)

    alphas = [0.0001, 0.001, 0.01, 0.1, 1.0, 10.0, 100.0, 1000.0]

    best_lasso = None
    best_lasso_mean_squared_error = float('inf')

    best_ridge = None
    best_ridge_mean_squared_error = float('inf')

    for alpha in alphas:
        lasso = Lasso(alpha=alpha, max_iter=10000)
        lasso.fit(X_training_scaled, y_training)
        predictions = lasso.predict(X_validation_scaled)
        mean_squared_errors = mean_squared_error(y_validation, predictions)

        if mean_squared_errors < best_lasso_mean_squared_error:
            best_lasso_mean_squared_error = mean_squared_errors
            best_lasso = lasso

    for alpha in alphas:
        ridge = Ridge(alpha=alpha)
        ridge.fit(X_training_scaled, y_training)
        predictions = ridge.predict(X_validation_scaled)
        mean_squared_errors = mean_squared_error(y_validation, predictions)

        if mean_squared_errors < best_ridge_mean_squared_error:
```

```
        best_ridge_mean_squared_error = mean_squared_errors
        best_ridge = ridge

print(f"Best Lasso alpha found: {best_lasso.alpha}")
print(f"Best Ridge alpha found: {best_ridge.alpha}\n")

print("--- Evaluation on Test Set ---")

models = {
    "Linear Regression": lr,
    f"Lasso Regression (alpha={best_lasso.alpha})": best_lasso,
    f"Ridge Regression (alpha={best_ridge.alpha})": best_ridge
}

for name, model in models.items():
    y_testing_prediction = model.predict(X_testing_scaled)

    mean_squared_errors = mean_squared_error(y_testing,
     y_testing_prediction)
    mean_absolute_errors = mean_absolute_error(y_testing,
     y_testing_prediction)

    print(f"{name}:")
    print(f"  Mean Squared Error (mean_squared_error) :
     {mean_squared_errors:,.2f}")
    print(f"  Mean Absolute Error (MAE): {mean_absolute_errors:,.2f}")
    print()
```

5. Final Analysis and Model Comparison

Linear Regression

This is the baseline model. It learns a direct linear relationship between the input features and the house valu: ax+b

Lasso Regression

Lasso adds L1 regularization, which penalizes the absolute values of the coefficients This may shrink some coefficients all the way to zero, so Lasso can also perform feature selection

Ridge Regression

Ridge adds L2 regularization,which penalizes the squared values of the coefficients It usually shrinks all coefficients smoothly and helps when features are highly correlated

Interpreting the metrics

- MSE: gives more weight to large errors because the errors are squared

- MAE: measures the average absolute difference between predicted and true values
- If Ridge performs slightly better, that usually means regularization helped reduce variance
- If all models perform similarly, then the basic linear model was already strong and not heavily overfitting

```
print("--- Final Analysis & Model Comparison ---")
print("""
1. Linear Regression: Acts as our baseline. It models the direct linear
         relationships
   between the features (like median income, rooms) and the house value.

2. Lasso Regression: Adds L1 regularization, which penalizes the absolute
         size of the coefficients.
   This can force some feature weights to exactly zero, effectively
         performing feature selection
   and creating a simpler, more interpretable model if some features are
         irrelevant.

3. Ridge Regression: Adds L2 regularization, which penalizes the squared size
         of the coefficients.
   This shrinks coefficients evenly and prevents any single feature from
         dominating the model,
   which is very helpful when there is multicollinearity (highly correlated
         features).

Comparison Results:
Depending on the outputted metrics, you'll generally notice:
- If all models perform extremely similarly, it indicates that the standard
         linear model wasn't
  overfitting heavily to begin with.
- Ridge often slightly outperforms pure Linear Regression in real-world dense
         datasets by reducing variance.
- Notice that mean_squared_error is mathematically much larger than
         mean_absolute_error. This is because mean_squared_error squares the
         errors
  before averaging them, meaning mean_squared_error heavily punishes large
         outlier mistakes (e.g., drastically mispricing).
""")
```

```
--- Final Analysis & Model Comparison ---

1. Linear Regression: Acts as our baseline. It models the direct linear
relationships
   between the features (like median income, rooms) and the house value.

2. Lasso Regression: Adds L1 regularization, which penalizes the absolute
size of the coefficients.
   This can force some feature weights to exactly zero, effectively
performing feature selection
   and creating a simpler, more interpretable model if some features are
irrelevant.

3. Ridge Regression: Adds L2 regularization, which penalizes the squared size
of the coefficients.
```

```
   This shrinks coefficients evenly and prevents any single feature from
dominating the model,
   which is very helpful when there is multicollinearity (highly correlated
features).

Comparison Results:
Depending on the outputted metrics, you'll generally notice:
- If all models perform extremely similarly, it indicates that the standard
linear model wasn't
  overfitting heavily to begin with.
- Ridge often slightly outperforms pure Linear Regression in real-world dense
datasets by reducing variance.
- Notice that mean_squared_error is mathematically much larger than
mean_absolute_error. This is because mean_squared_error squares the errors
  before averaging them, meaning mean_squared_error heavily punishes large
outlier mistakes (e.g., drastically mispricing).
```

6. Main Function

This final section:

1. Reads the dataset
2. Splits it into train, validation, and test sets
3. Trains the models
4. Prints the evaluation results

```python
def main():
    df = read_California_dataset()
    X_training, X_validation, X_testing, y_training, y_validation, y_testing
         = split_data(df)
    train_and_evaluate_models(X_training, y_training, X_validation,
         y_validation, X_testing, y_testing)

if __name__ == "__main__":
    main()
```

```
--- Tuning Models on Validation Set ---
Best Lasso alpha found: 10.0
Best Ridge alpha found: 1.0

--- Evaluation on Test Set ---
Linear Regression:
  Mean Squared Error (mean_squared_error) : 4,661,701,996.80
  Mean Absolute Error (MAE): 49,688.84

Lasso Regression (alpha=10.0):
  Mean Squared Error (mean_squared_error) : 4,661,437,350.75
  Mean Absolute Error (MAE): 49,697.52

Ridge Regression (alpha=1.0):
  Mean Squared Error (mean_squared_error) : 4,661,503,722.14
```

Mean Absolute Error (MAE): 49,692.74